

CS & IT ENGINEERING



Data Structure & Programming

Trees and Graphs

Lecture No.- 06



By- Satya sir

Recap of Previous Lecture



- Deletion from Binary Heap

- Heap Orderings

$$T(N) = \sum_{L=1}^{N-1} T(L) * T(R)$$



Topics to be Covered



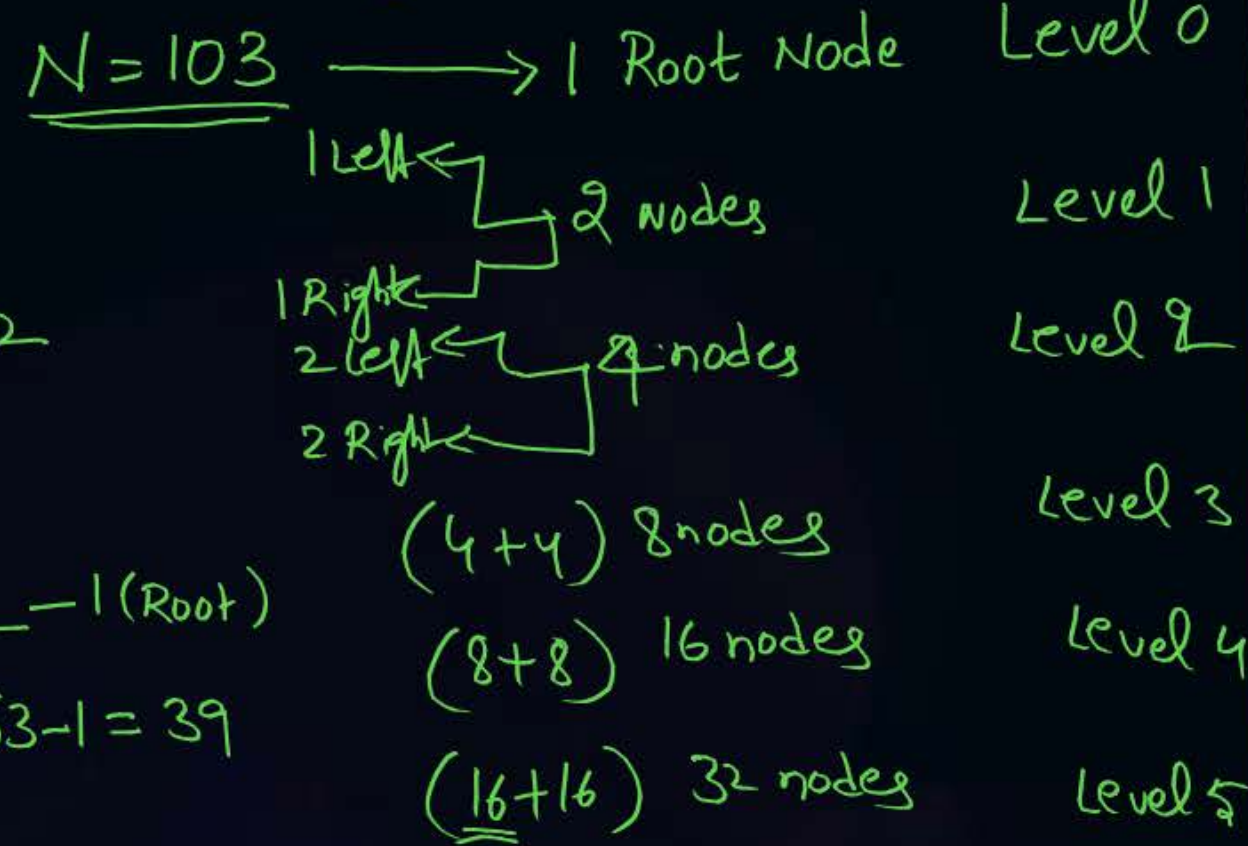
- Heap orderings
- Time Complexity of Heap
- Time Complexity of BST
- Balance factor
- AVL Tree



Topic : AVL Trees - 1

Example of Heap Orderings

$$T(N) = {}^{N-1}C_L * T(L) * T(R)$$



$$L = 31 + 32 = 63$$

$$R = N - L - 1(\text{Root}) = 103 - 63 - 1 = 39$$

Total Right subtree nodes (OR)
Total Left subtree nodes = 1 + 2 + 4 + 8 + 16 = 31
[Except last level]

$$T(103) = {}^{102}_{63} C * T(63) * T(39)$$

$$1(\text{Root}) + 31(\text{Left}) + 31(\text{Right}) \text{ Except last level}$$

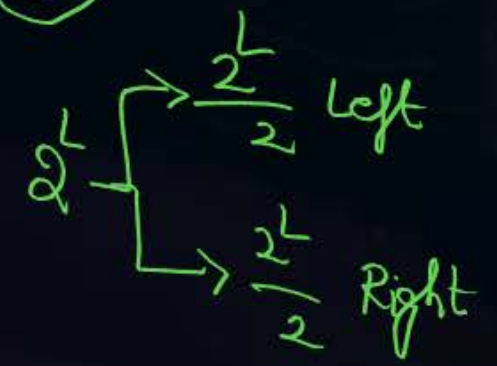
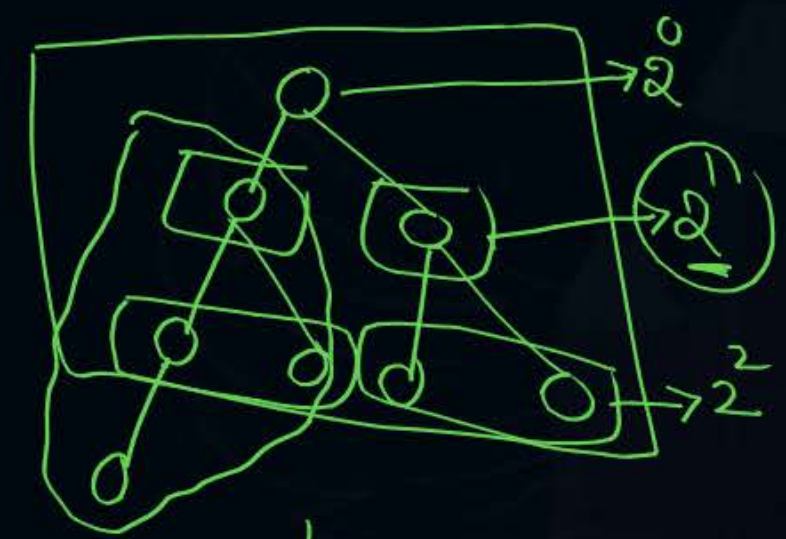
⇒ 63 nodes

⇒ Remaining nodes = 103 - 63 = 40 nodes for last level.



minimum height, (or) Level $N = 2^{(h+1)} - 1$
 $\Rightarrow 2^{(h+1)} = N + 1$

At Level 'L' ⇒ 2^L Nodes $h+1 = \log^{N+1}_2$



$$h = \left\lceil \log_2^{N+1} \right\rceil - 1$$

$$= \left\lceil \log_2^{103+1} \right\rceil - 1$$

$$= \left\lceil \log_2^{104} \right\rceil - 1$$

$$= 7 - 1 = 6 \rightarrow \text{Left Nodes} = 2^h - 1 = 63$$



Topic : AVL Trees - 1

① $N = 200$

- 1 Root node = 1
- Left = 63 + 64 Last level = 127
- Right = 63 + 9 (last level) = 72

$h = 7$

$$T(200) = {}^{199}_C * T(127) * T(72)$$

② $N = 70$

- 1 Root node = 1
- Left = 31 + 7 nodes (last level) = 38
- Right = 31 = 31

$h = 6$

$$T(70) = {}^{69}_C * T(38) * T(31)$$

③ $N = 314$

- 1 Root node = 1
- Left: 127 nodes + 59 nodes = 186
- Right: 127 nodes = 127

$$T(314) = {}^{313}_C * T(186) * T(127)$$

$$h = \left\lceil \log_2(N+1) \right\rceil - 1$$

$$= 9 - 1 = 8$$



Topic : AVL Trees - 1



Time Complexity of Binary Heap

Insertion of a node : $O(\log_2^n)$

Deletion of a node : $O(\log_2^n)$

Build/Construction of Binary Heap : $O(n * \log_2^n)$

Heapification : $O(\log_2^n)$





Topic : AVL Trees - 1

Time Complexity : $\begin{cases} \rightarrow O(1) \text{ Best case} \\ \rightarrow O(\log n) \text{ Avg case} \\ \rightarrow O(n) \text{ Worst case} \end{cases}$

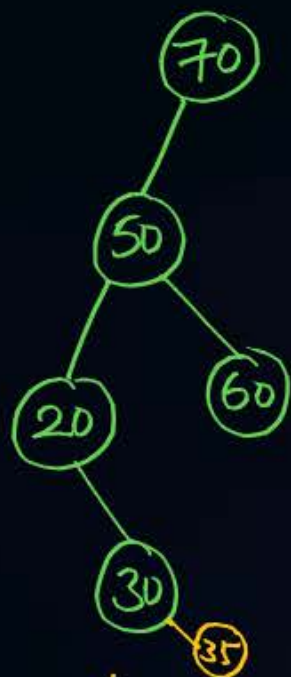
5 Nodes
 (Counted from 1) $\begin{cases} \rightarrow \text{min height} = 3 \\ \rightarrow \text{Max height} = 5 \end{cases}$



BST Construction (Let us Count height from 1)

Insertion Sequences

Seq:1 70, 50, 60, 20, 30

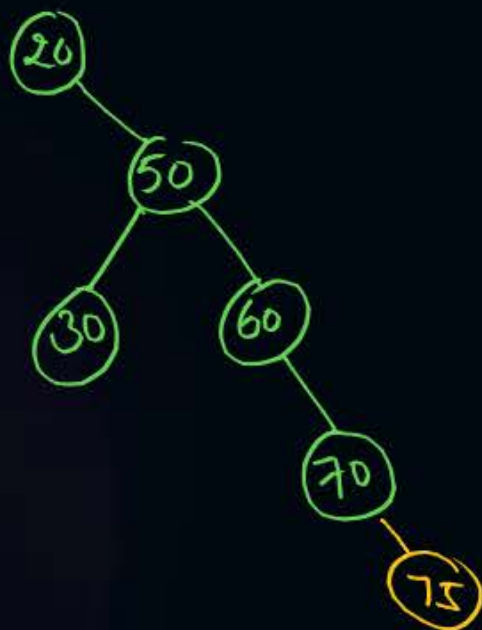


Search
key=30
Compare with 70, 50, 20, 30

height = 4 = No. of Comparisons

Insert 35: $35 < 70$, $35 < 50$, $35 > 20$,
 $35 > 30$

Seq:2 20, 50, 30, 60, 70

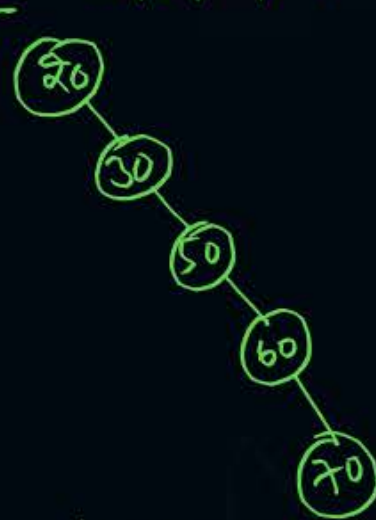


height = 4 = No. of Comp

key = 70

Insert 75: 4 Comparisons

Seq:3 20, 30, 50, 60, 70



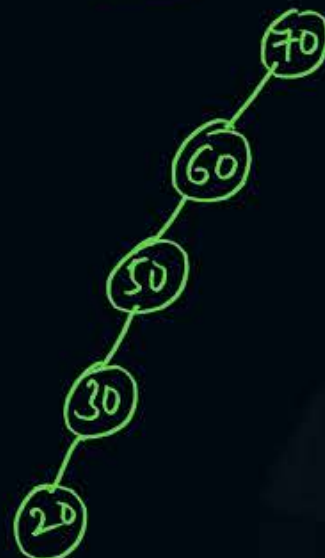
key = 70

height = 5

No. of Comp = 5

Insert 75: 5 Comp

Seq:4 70, 60, 50, 30, 20



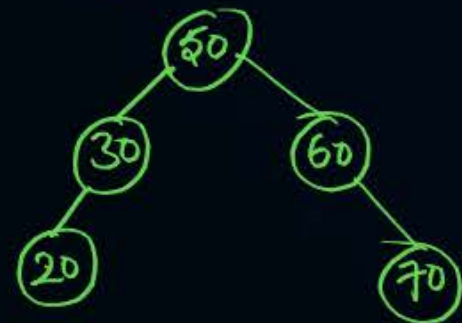
key = 20

height = 5

Comp = 5

Insert 25: Comp = 5

Seq:5 50, 30, 60, 20, 70



key = 70

height = 3

No. of Comp = 3

Insert 75 or 25
3 Comp



Topic : AVL Trees - 1

- The Time complexity of BST is Proportionate to height of BST.
- So, To retain / maintain the time complexity to $O(\log_2 n)$ always, height need to be balanced.

- Height Balanced BST is called as AVL Tree.

→ Balance Factor of each node must be $= \{-1, 0, +1\}$

→ Balance Factor = Height of Left Subtree - Height of Right Subtree of a Node

NOTE: Height Numbered from 1, Balance Factor of any leaf node = 0



Ex:

Ex:



Ex:

Ex:

Ex:



Ex:



Topic : AVL Trees - 1



Q. How To Balance a BST ?

Ans: By Rotations

4 Rotations

① LL Rotation

② RR Rotation

③ LR Rotation

④ RL Rotation



2 mins Summary



- Heap orderings
- Time Complexity of Heap
- BST Time Complexity
- Need for Balancing
- Balance Factor
- Rotations Intro

To be contd ...





THANK - YOU